

# SKILL-6

Roll No. : 2520030437

Name : Bandla Venkata Rama Vinay

SEC-8A

## SKILL-6:

**Aim:** Develop C programs demonstrating command execution through a child process and processing of escape characters in user input.

The first program creates a child process using **fork()**, executes the **ls -l** command using **execlp()**, and makes the parent wait for the child using **waitpid()**. The second program processes an input string and removes the escape character before producing the parser output.

---

### Procedure :

1. Create the process-execution program in **Skill-6-1.c**.
2. Create a child process using **fork()** and check for fork failure.
3. In the child process, display its PID and execute **ls -l** using **execlp()**.
4. In the parent process, display the parent PID and wait for the child using **waitpid()**.
5. Create the escape-character processing program in **Skill-6.c**.
6. Read the input, preserve the original input, process a backslash followed by a character, and display the processed output.

## Output - Process Creation and Command Execution :

```
Skill > Skill-6.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char input[200];
7
8      printf("Enter input with escape characters: ");
9      fgets(input, sizeof(input), stdin);
10
11     input[strcspn(input, "\n")] = '\0';
12
13     printf("\nOriginal Input: %s\n", input);
14
15     printf("\nProcessed Output:\n");
16
17     for (int i = 0; input[i] != '\0'; i++)
18     {
19         if (input[i] == '\\' && input[i + 1] != '\0')
20         {
21             i++;
22             printf("%c", input[i]);
23         }
24         else
25         {
26             printf("%c", input[i]);
27         }
28     }
29
30     printf("\n\nParser output validated successfully.\n");
31
32     return 0;
33 }
```

The submitted execution shows the parent process with PID **1709** waiting for the child. The child process has PID **1710** and executes **ls -l**, producing the directory listing. After the command completes, the parent reports that the child process completed.

## Code - Process Creation and Execution:

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_6
Enter input with escape characters: hello\ world

Original Input: hello\ world

Processed Output:
hello world

Parser output validated successfully.
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ |
```

## Output - Escape Character Processing :

```
Skill > c Skill-6-1.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      pid_t pid;
9      int status;
10
11     pid = fork();
12
13     if (pid < 0)
14     {
15         perror("fork failed");
16         return 1;
17     }
18
19     if (pid == 0)
20     {
21         printf("Child Process\n");
22         printf("Child PID = %d\n", getpid());
23
24         execlp("ls", "ls", "-l", NULL);
25
26         perror("Execution failed");
27         exit(1);
28     }
29     else
30     {
31         printf("Parent Process\n");
32         printf("Parent PID = %d\n", getpid());
33         printf("Waiting for child...\n");
34
35         waitpid(pid, &status, 0);
36
37         printf("Child process completed.\n");

```

The submitted execution enters **hello\ world**. The program displays the original input and then processes the backslash so that the resulting output is **hello world**. It finally reports that the parser output was validated successfully.

## Code - Escape Character Processing:

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_6-1
Parent Process
Parent PID = 1709
Waiting for child...
Child Process
Child PID = 1710
total 118
-rwxrwxrwx 1 vinay vinay 1663 Aug 24 08:19 Skill-3.c
-rwxrwxrwx 1 vinay vinay 857 Aug 24 08:28 Skill-4-1.c
-rwxrwxrwx 1 vinay vinay 524 Aug 24 08:24 Skill-4.c
-rwxrwxrwx 1 vinay vinay 966 Aug 24 08:33 Skill-5.c
-rwxrwxrwx 1 vinay vinay 715 Aug 24 08:38 Skill-6-1.c
-rwxrwxrwx 1 vinay vinay 652 Aug 24 08:36 Skill-6.c
-rwxrwxrwx 1 vinay vinay 16392 Aug 24 08:21 skill_3
-rwxrwxrwx 1 vinay vinay 16216 Aug 24 08:24 skill_4
-rwxrwxrwx 1 vinay vinay 16224 Aug 24 08:29 skill_4-1
-rwxrwxrwx 1 vinay vinay 16264 Aug 24 08:34 skill_5
-rwxrwxrwx 1 vinay vinay 16224 Aug 24 08:37 skill_6
-rwxrwxrwx 1 vinay vinay 16304 Aug 24 08:39 skill_6-1
Child process completed.
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ |
```

## Functions and Program Operations

| Function / Operation | Purpose                                                      |
|----------------------|--------------------------------------------------------------|
| fork()               | Creates a child process from the parent process.             |
| getpid()             | Returns the PID of the current process.                      |
| execlp()             | Replaces the child process image with the requested command. |
| waitpid()            | Makes the parent wait for the specified child process.       |
| fgets()              | Reads input from standard input.                             |
| strcspn()            | Locates the newline character so it can be removed.          |

### Observation :

- The first program creates a parent-child process relationship using **fork()**.
- The child process displays PID 1710 and executes **ls -l**.
- The parent process displays PID 1709 and waits for the child using **waitpid()**.
- The second program reads an input containing an escape character and displays both the original and processed forms.
- The input **hello\ world** is processed into **hello world**.
- The parser output is reported as validated successfully.

### Result :

The SKILL-6 programs successfully demonstrate process creation and command execution using **fork()**, **execlp()**, and **waitpid()**, along with escape-character processing in command input. Both submitted executions completed successfully with the expected outputs.

**SKILL-6 Completed Successfully**